

0602_연습문제 과제_박수민

p.259

1. 다음 중 char형 배열이 바르게 초기화된 것은 O, 그렇지 않은 것은 X를 표시하세요.

- 1) char str[80] = {'p', 'i', 'g'}; (O)
- 2) char str[] = "elephant;" (O)
- 3) char str[5] = "apple"; (X)
- 4) char str[2] = {"sun", "moon"}; (X)

2. 다음중 널 문자의 상수 표현법으로 옳은 것을 고르세요.

- 1) NULL
- 2) /0
- 3) '0'
- 4) '\0'

A: (4)

3. 다음 코드는 2개의 문자열을 입력받아 위치를 바꾼 후에 출력합니다. 빈칸을 채워 프로그램을 완성

...

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str1[80], str2[80];
    char temp[80];

    printf("두 문자열 입력: ");
    scanf("%s %s", str1, str2);
    printf("바꾸기 전: %s %s\n", str1, str2);

    strcpy(temp, str1); // temp에 str1 복사
    strcpy(str1, str2); // str1에 str2 복사
    strcpy(str2, temp); // str2에 temp 복사
    printf("바꾼 후: %s %s\n", str1, str2);
```

```
    return 0;
}
```

p. 274

1. 다음 각 변수의 주소를 저장할 포인터를 선언하세요.(변수명은 p로 합니다.)

1) char ch;

A: char *pch;

2) int in;

A: int *pin

3) double db;

A: double *pdb;

2. 다음과 같이 변수가 초기화되고 메모리에 할당되었다고 가정하고 주소 연산자와 간접 참조 연산자를 사용한 수식의 결과값을 적으세요.

...

char ch = 'A';

int in = 10;

double db = 3.4;

...

수식	&ch	&in	&db	*&ch	*&in	*&db
결과값	100	101	105	A	10	3.4

p.288

1. 다음 코드를 참고해 보기에서 상수와 변수를 구분하세요.

...

```
int a = 10;
int *p = &a;
*p = 20;
...
```

1) a: 변수, 2) 10: 상수, 3) p: 변수, 4) *p: 변수 5) &a: 상수

2. 주소 값의 크기가 4바이트일 때, sizeof 연산의 결과값이 가장 큰 것을 고르세요.

...

```
char *pc;
double *pd;
...
```

A: (4) sizeof(*pd) -> 8byte

3. 다음 코드의 실행결과를 적으세요.

...

```
int a = 10, b = 20;
int *pa = &a, *pb = &b, *pt;
pt = pa;
pa = pb;
pb = pt;
printf("%d, %d", *pa, *pb);
...
```

실행결과: 20 10

p.289

도전 실전 예제- 미니 정렬 프로그램

키보드로 실수 3개를 입력한 후 큰 숫자부터 작은 숫자로 정렬한 뒤 출력하는 프로그램을 작성합니다. 다음 코드와 출력 결과를 참고해 line_up 함수를 작성하세요.

line_up 함수에는 정의된 swap 함수를 호출해 구현하세요.

...

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
void swap(double* pa, double* pb);
```

```
void line_up(double* maxp, double* midp, double *minp);
```

```
int main()
```

```
{
```

```
    double max, mid, min;
```

```
    printf("실수값 3개 입력: ");
```

```
    scanf("%lf %lf %lf", &max, &mid, &min);
```

```
    line_up(&max, &mid, &min);
```

```
    printf("정렬된 값 출력: %.1lf %.1lf %.1lf\n", max, mid, min);
```

```
    return 0;
```

```
}
```

```
void swap(double* pa, double* pb)
```

```
{
```

```
    double temp;
```

```
    temp = *pa; // temp에 pa가 가리키는 값 저장
```

```
    *pa = *pb; // pa가 가리키는 값에 pb가 가리키는 값 저장
```

```
    *pb = temp; // pb가 가리키는 값에 temp 저장
```

```
}
```

```
void line_up(double* maxp, double* midp, double *minp)
```

```
{
```

```
    if (*maxp < *midp) {  
        swap(maxp, midp);
```

```
    }
```

```
    if (*maxp < *minp) {  
        swap(maxp, minp);
```

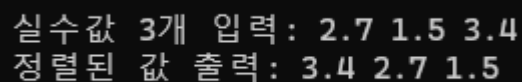
```
    }
```

```
    if (*midp < *minp) {  
        swap(midp, minp);
```

```
    }
```

```
}
```

```
...
```



```
실수값 3개 입력: 2.7 1.5 3.4  
정렬된 값 출력: 3.4 2.7 1.5
```

p.307~308 확인문제

다음과 같이 배열과 포인터가 초기화되고 그림처럼 메모리에 할당되었다고 가정
...

```
double ary[5] = { 1.2, 3.5, 7.4, 0.5, 10.0 };
```

```
double *pa = ary;
```

```
double *pb = ary + 2;
```

...

ary 배열

100 108 116 124 132

1.2	3.5	7.4	0.5	10.0
-----	-----	-----	-----	------

1. 다음 각 항목의 값을 생각해 적으세요.

1) ary -> 100

2) *(ary +1) -> 3.5

3) pa + 2 -> 116

4) pa[3] -> 0.5

5) *pb -> 7.4

6) pb - pa -> 2

2. 다음중 사용법이 맞는 것은 O, 잘못된 표현은 X를 표시하세요.

1) ary[5] -> (X)

2) ary++ -> (X)

3) ++(*ary)->(O)

4) pb[-2] -> (O)

5) *(pb + 3) -> (X)

6) *(++pa) -> (O)

3. 다음은 포인터 pb로 세 번째 배열 요소부터 마지막 배열 요소까지 출력하는 코드입니다.
코드가 완성되도록 빈칸을 채우세요.

...

```
for (i=0; i<3; i++)
```

```
{
```

```
    printf("%.1lf ", *pb);
```

```
    pb++;
```

```
}
```

...

p.319

1. 다음과 같이 선언된 함수가 있을 때 함수의 호출이 적합하지 않은 것을 고르세요.

```
void func(int *p);
```

```
1) int ary[5];  
func(ary);
```

```
2) int ary[] = {1, 2, 3};  
func(ary);
```

```
3) double ary[5];  
func(ary);
```

```
4) int ary[5];  
func(ary + 2);
```

A: (3)

2. 다음은 각 달의 일수를 저장한 배열입니다. month[0]은 1월이며, 첨자가 순차적으로 증가할 때 월도 함께 증가합니다. month 배열의 값을 한 줄에 5개씩 출력하는 함수를 작성하세요.
...

```
int month[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

```
void print_month(int *mp)  
{  
    int i;  
    for (i=0; i<12; i++)  
    {  
        printf("%5d", mp[i]);  
        if (i%5==0) {  
            printf("\n");  
        }  
    }  
}
```

p.320

1~45 중에 서로 다른 수를 배열에 입력하고 출력.

입력한 수가 이미 저장된 수와 같으면 에러 메시지를 출력하고 다시 입력.

다음 함수의 선언과 정의를 참고해 빈 부분을 채워 완성하기

```

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void input_nums(int* lotto_nums); // 배열에 로또 번호를 입력하는 함수
```

```
void print_nums(int* lotto_nums); // 배열에 저장된 로또 번호를 출력하는 함수
```

```
int main(void)
```

```
{
```

```
 int lotto_nums[6]; // 로또 번호를 저장할 배열
```

```
 input_nums(lotto_nums); // 로또 번호 입력
```

```
 print_nums(lotto_nums); // 로또 번호 출력
```

```
 return 0;
```

```
}
```

```
void input_nums(int* lotto_nums)
```

```
{
```

```
 int i, j, num;
```

```
 for (i = 0; i < 6; i++)
```

```
 {
```

```
 printf("번호 입력 (1~45): ");
```

```
 scanf("%d", &num);
```

```
 if (num < 1 || num > 45)
```

```
 {
```

```
 printf("잘못된 번호입니다. 다시 입력하세요.\n");
```

```
 i--;
```

```
 continue;
```

```
 }
```

```
 for (j = 0; j < i; j++)
```

```
 {
```

```
 if (lotto_nums[j] == num)
```

```
 {
```

```
 printf("같은 번호가 있습니다!\n");
```

```
 i--;
```

```
 break;
```

```
 }
```

```
 }
```

```
 if (j == i)
```

```
 {
```

```
 lotto_nums[i] = num;
```

```
 }
```

```
 }
```

```
}
```

```
void print_nums(int* lotto_nums)
```

```
{
```

```
 int i;
```

```
 printf("로또 번호: ");
```

```

 for (i = 0; i < 6; i++)
 {
 printf("%d ", lotto_nums[i]);
 }
 printf("\n");
}
...

```

출력결과

```

번호 입력 (1~45): 3
번호 입력 (1~45): 7
번호 입력 (1~45): 15
번호 입력 (1~45): 3
같은 번호가 있습니다!
번호 입력 (1~45): 22
번호 입력 (1~45): 35
번호 입력 (1~45): 40
로또 번호: 3 7 15 22 35 40

```

p.331

1. 다음 중 값이 같은 것을 모두 고르세요. (아스키 코드 표 참고)

- |                      |             |          |
|----------------------|-------------|----------|
| 1) 'a'               | 2) 'A'-32   | 3) 'b'-1 |
| 4) 'A' + ('b' - 'B') | 5) 97 - '0' | 6) 97    |

풀이) (1) 97 (2) 33 (3) 97 (4) 97 (5) 49 (6) 97

A: 1, 3, 4, 6

2. 키보드로 입력받은 문자의 아스키 코드 값을 출력하는 프로그램이 완성되도록 빈칸을 채우세요.

...

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

```

```

int main() {
 char ch;

 printf("문자 입력: ");
 scanf("%c", &ch);
 printf("%c 문자의 아스키 코드 값: %d\n", ch, ch);

 return 0;
}
...

```

```

문자 입력: Q
Q 문자의 아스키 코드 값: 81

```



p.342

1. 키보드로 데이터를 입력할 때 버퍼에 남아 있는 데이터를 모두 지우는 방법으로 적당한 것을 고르세요.

1) fflush(stdout);

2) while (getchar() != '\n');

3) fgetc(stdin);

4) scanf("%c", &ch);

A: (2)

2. 키보드로부터 하나의 문자를 입력받는 방법 중 적절하지 못한 것을 고르세요.

...

char ch;

int num;

...

1) num = getchar();

2) ch = scanf("%c");

3) ch = fgetc(stdin);

4) scanf("%c", &ch);

5) getchar(&ch);

6) scanf("%c", &num);

A: (2), (5), (6)

3. 다음의 키보드로 입력한 문장에서 소문자의 개수를 세어 출력하는 프로그램이 완성될 수 있도록 빈칸을 채우세요.

...

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
 int ch;
```

```
 int cnt = 0;
```

```
 ch = getchar();
```

```
 while (ch != '\n')
```

```
 {
```

```
 if ((ch >= 'a') && (ch <= 'z')) {
 cnt++;
```

```
 }
```

```
 ch = getchar();
```

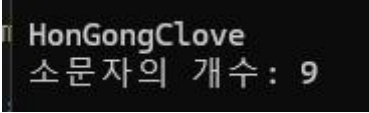
```
 }
```

```
 printf("소문자의 개수: %d\n", cnt);
```

```
 return 0;
```

```
}
```

...



```
HonGongClove
소문자의 개수 : 9
```

p.343

길이가 가장 긴 단어 찾기

키보드로 입력한 단어 중에서 길이가 가장 긴 단어의 길이를 출력합니다.

한 줄에 하나의 단어만 입력하며

ctrl + z를 누를때까지 입력한 단어 중에서 길이가 가장 긴 단어의 길이를 출력합니다.

배열은 사용하지 않고 작성합니다.

...

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
 char ch;
```

```
 int len, max = 0;
```

```
 while (1)
```

```
 {
```

```
 ch = getchar();
```

```
 if (ch == -1) break;
```

```
 len = 0;
```

```
 while (ch != '\n')
```

```
 {
```

```
 len++;
```

```
 ch = getchar();
```

```
 }
```

```
 //printf("길이: %d\n", len);
```

```
 if (len > max)
```

```
 {
```

```
 max = len;
```

```
 }
```

```
 printf("\n");
```

```
 }
```

```
 printf("가장 긴 단어의 길이: %d\n", max);
```

```
 return 0;
```

```
}
```

...

출력결과

```
apple
```

```
kiwi
```

```
strawberry
```

```
mango
```

```
^Z
```

```
가장 긴 단어의 길이: 10
```